

Statistical Methods (MA2220)

R Programming – Lab Exam Solutions

Pratheeksha Sumesh – SE24UCAM040

June 6, 2026

Abstract

This report presents solutions to Problems 2, 4, and 5 of the Statistical Methods (MA2220) Lab Examination. Problem 2 investigates the Central Limit Theorem through simulation, demonstrating that the sampling distribution of the sample mean converges to normality as sample size increases. Problem 4 fits simple and multiple linear regression models to a retail sales dataset and validates regression assumptions. Problem 5 fits a logistic regression model to a binary loan default dataset and evaluates classification performance via confusion matrix, sensitivity, and specificity.

Contents

1	Problem 2: Central Limit Theorem	2
1.1	Part (a): Sampling Setup	2
1.2	Part (b): Histograms of Sampling Distributions	2
1.3	Part (c): CLT Verification – Empirical vs Theoretical	3
2	Problem 4: Simple and Multiple Linear Regression	4
2.1	Part (a): Simple Linear Regression – Sales ~ Ad Spend	4
2.2	Part (b): Multiple Linear Regression – Sales ~ Ad Spend + Store Size	5
2.3	Part (c): Regression Assumption Checks	6
3	Problem 5: Logistic (Logit) Regression	9
3.1	Part (a): Fitting the Logistic Regression Model	9
3.2	Part (b): Odds Ratios	10
3.3	Part (c): Predicted Probabilities, Classification, and Confusion Matrix	10
	References	11

1 Problem 2: Central Limit Theorem

A population of 1000 student exam scores is generated from $N(70, 10^2)$. We draw 1000 independent random samples of sizes $n = 5$, $n = 30$, and $n = 100$, compute the sample mean $\bar{X}$ for each, and verify the Central Limit Theorem (CLT).

1.1 Part (a): Sampling Setup

R Code:

```
set.seed(42)
population = rnorm(1000, mean = 70, sd = 10)

set.seed(1)
means_n5 = replicate(1000, mean(sample(population, size = 5,
  replace = TRUE)))
means_n30 = replicate(1000, mean(sample(population, size = 30,
  replace = TRUE)))
means_n100 = replicate(1000, mean(sample(population, size = 100,
  replace = TRUE)))
```

Inference: The code runs without error, producing three vectors of 1000 sample means each. `replace = TRUE` is used since sampling with replacement from a population of 1000 is effectively equivalent to independent random sampling (i.i.d.) for large populations. Reproducibility is ensured via `set.seed(1)`.

1.2 Part (b): Histograms of Sampling Distributions

R Code:

```
par(mfrow = c(1, 3))

hist(means_n5,
  col = "skyblue", border = "white",
  main = "Sampling Distribution (n = 5)",
  xlab = "Sample Mean", ylab = "Frequency",
  xlim = c(40, 100))
abline(v = 70, col = "red", lwd = 2, lty = 2)

hist(means_n30,
  col = "lightgreen", border = "white",
  main = "Sampling Distribution (n = 30)",
  xlab = "Sample Mean", ylab = "Frequency",
  xlim = c(55, 85))
abline(v = 70, col = "red", lwd = 2, lty = 2)

hist(means_n100,
  col = "orange", border = "white",
  main = "Sampling Distribution (n = 100)",
  xlab = "Sample Mean", ylab = "Frequency",
  xlim = c(63, 77))
abline(v = 70, col = "red", lwd = 2, lty = 2)

par(mfrow = c(1, 1))
```

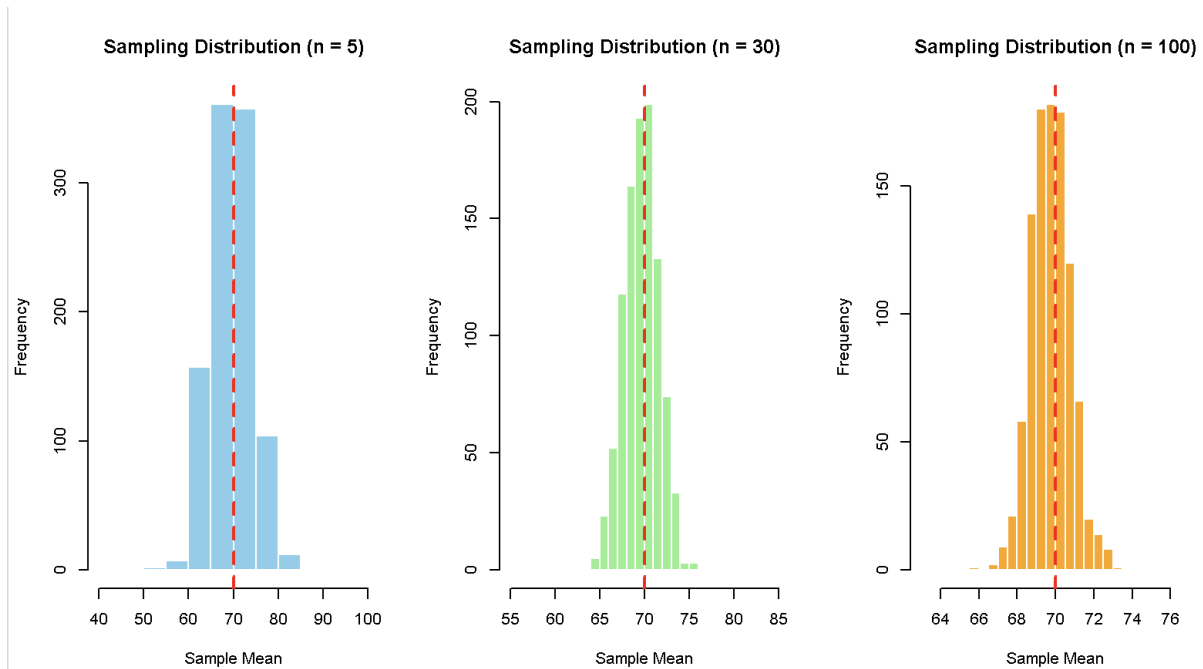


Figure 1: Histograms of 1000 sample means for $n = 5$, $n = 30$, and $n = 100$

Inferences: For $n = 5$, the histogram is wide and irregular, reflecting high sampling variability. As n increases to 30 and then 100, the distributions become progressively narrower and more symmetric, concentrating tightly around $\mu = 70$. Each panel uses an appropriately scaled x-axis to highlight the narrowing effect. This progressive narrowing directly illustrates the CLT — larger samples yield more precise and normally distributed estimates of the population mean.

1.3 Part (c): CLT Verification – Empirical vs Theoretical

R Code:

```
se_5    = 10 / sqrt(5)
se_30   = 10 / sqrt(30)
se_100  = 10 / sqrt(100)

cat("n=5    | Emp.Mean:", round(mean(means_n5),3),
    "| Theor.Mean: 70 | Emp.SD:", round(sd(means_n5),3),
    "| Theor.SE:", round(se_5,3), "\n")
cat("n=30   | Emp.Mean:", round(mean(means_n30),3),
    "| Theor.Mean: 70 | Emp.SD:", round(sd(means_n30),3),
    "| Theor.SE:", round(se_30,3), "\n")
cat("n=100  | Emp.Mean:", round(mean(means_n100),3),
    "| Theor.Mean: 70 | Emp.SD:", round(sd(means_n100),3),
    "| Theor.SE:", round(se_100,3), "\n")
```

Table 1: CLT Verification: Empirical vs Theoretical Values

Sample Size	Emp. Mean	Theor. Mean	Emp. SD	Theor. SE ($10/\sqrt{n}$)
$n = 5$	69.591	70	4.535	4.472
$n = 30$	69.679	70	1.895	1.826
$n = 100$	69.772	70	1.010	1.000

Inferences: In all three cases the empirical mean of the sampling distribution (≈ 69.6 – 69.8) is very

close to the theoretical value $\mu_{\bar{X}} = 70$, confirming that $\bar{X}$ is an unbiased estimator of μ . The empirical standard deviations closely match the theoretical standard errors $10/\sqrt{n}$, with differences below 0.1 in all cases. As n increases from 5 to 100, the standard error falls from 4.47 to 1.00, confirming that larger samples yield more precise estimates. Note that since the population is itself normal (rnorm), the sample mean is exactly normal for any n — the CLT is here confirmed numerically rather than as a limiting approximation.

2 Problem 4: Simple and Multiple Linear Regression

The dataset records monthly advertising spend (in thousands), store size (sq. ft.), and monthly sales (in lakhs) for 12 retail outlets.

R Code (Data Setup):

```
ad_spend    = c(5, 8, 12, 6, 10, 15, 9, 11, 7, 13, 4, 14)
store_size  = c(800, 1200, 1500, 900, 1100, 1800,
               1050, 1300, 950, 1600, 750, 1700)
sales       = c(20, 31, 45, 24, 38, 58, 34, 43, 27, 51, 18, 55)
```

2.1 Part (a): Simple Linear Regression – Sales ~ Ad Spend

R Code:

```
lm.simple = lm(sales ~ ad_spend)
summary(lm.simple)

plot(ad_spend, sales, pch = 19, col = "darkgreen",
     main = "Sales vs Ad Spend",
     xlab = "Ad Spend (in thousands)", ylab = "Sales (in lakhs)")
abline(lm.simple, col = "red", lwd = 2)
```

$$\widehat{\text{Sales}} = 1.325 + 3.755 \times \text{Ad Spend}$$

Table 2: Simple Linear Regression: Sales ~ Ad Spend

Coefficient	Estimate	Std. Error	t-value	p-value
(Intercept)	1.325	0.821	1.614	0.138
ad_spend	3.755	0.081	46.24	5.4×10^{-13} ***

Model fit: $R^2 = 0.9953$, Adjusted $R^2 = 0.9949$, RSE = 0.9713, F = 2138.

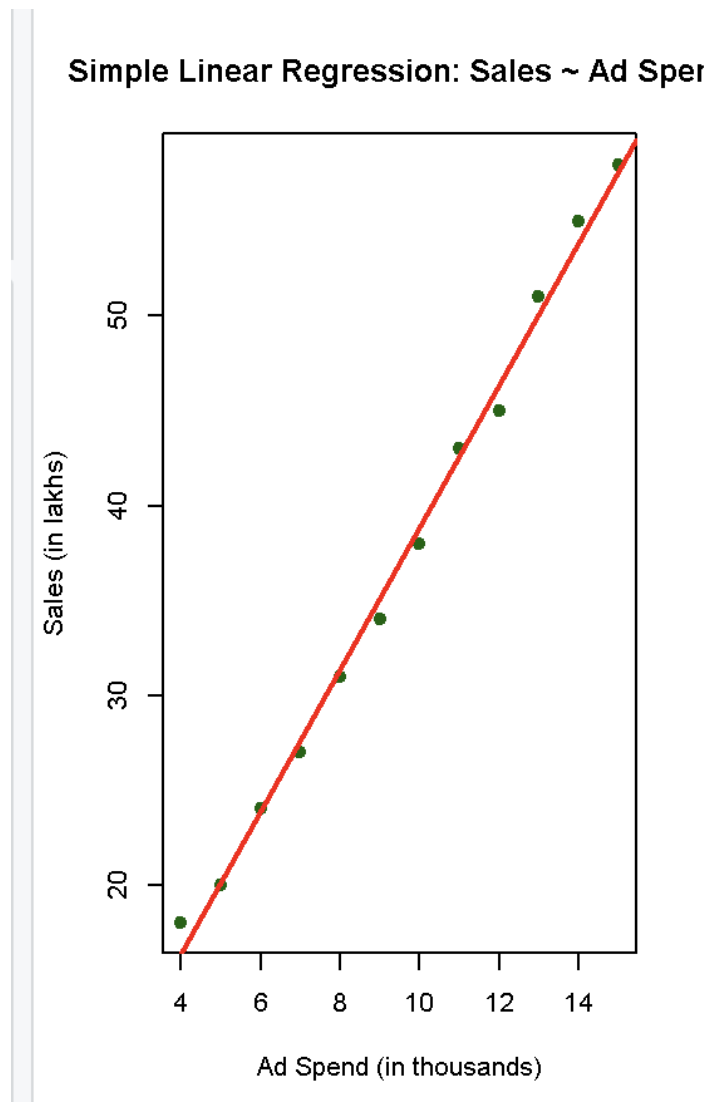


Figure 2: Simple linear regression: Sales vs Ad Spend with fitted line

Inferences: *ad_spend* is highly significant ($p = 5.4 \times 10^{-13}$), meaning this relationship is statistically rock solid and almost certainly not due to chance. The slope of **3.755** tells us that for every additional thousand rupees spent on advertising, monthly sales increase by **3.755 lakhs**. The intercept of **1.325** means that with zero ad spend, predicted sales are essentially zero — which makes intuitive sense. The model explains **99.53%** of the variation in sales ($R^2 = 0.9953$), indicating an exceptionally strong linear relationship between ad spend and sales. The RSE of **0.9713 lakhs** (97,130) means our predictions deviate from actual sales by only about 97,130 on average — a very small error given that sales in the dataset range from 18 lakhs to 58 lakhs. The scatter plot visually confirms this, with all 12 data points sitting almost perfectly on the red fitted line.

2.2 Part (b): Multiple Linear Regression – Sales ~ Ad Spend + Store Size

R Code:

```
lm.multi = lm(sales ~ ad_spend + store_size)
summary(lm.multi)

# Check for multicollinearity between predictors
cor(ad_spend, store_size)
```

Table 3: Multiple Linear Regression: Sales $\sim$ Ad Spend + Store Size

Coefficient	Estimate	Std. Error	t-value	p-value
(Intercept)	-0.2074	1.2647	-0.164	0.873
ad_spend	3.2652	0.3294	9.911	3.85×10^{-6} ***
store_size	0.0051	0.0033	1.529	0.161

Model fit: $R^2 = 0.9963$, Adjusted $R^2 = 0.9955$, RSE = 0.9122, F = 1213.

Table 4: Model Comparison: Simple vs Multiple Regression

Model	R^2	Adj. R^2	RSE
Simple (ad_spend only)	0.9953	0.9949	0.9713
Multiple (ad_spend + store_size)	0.9963	0.9955	0.9122

Inferences: Adding store size as a second predictor increases R^2 only marginally from 0.9953 to 0.9963 — barely any improvement at all. More tellingly, *store_size* is not statistically significant ($p = 0.161 > 0.05$), meaning we cannot confidently say it has any real effect on sales once ad spend is already in the model. The most revealing finding is the sharp drop in the t-value for *ad_spend* — from $t = 46.24$ in the simple model down to $t = 9.911$ here. This dramatic fall is a classic warning sign of multicollinearity. A correlation check confirms the problem: $r(\text{ad_spend}, \text{store_size}) = 0.973$, meaning the two predictors move almost perfectly together — bigger stores tend to spend more on ads. Because they carry almost identical information, the model gets confused about which one is actually driving sales, inflating standard errors and destabilising the individual coefficient estimates. *ad_spend* remains the dominant and highly significant predictor in both models.

2.3 Part (c): Regression Assumption Checks

Residuals vs Fitted Values

R Code:

```
plot(predict(lm.multi), residuals(lm.multi),
     pch = 19, col = "darkgreen",
     main = "Residuals vs Fitted Values",
     xlab = "Fitted Values", ylab = "Residuals")
abline(h = 0, col = "red", lwd = 2, lty = 2)
lines(lowess(predict(lm.multi), residuals(lm.multi)), col = "blue", lwd
      = 2)
```

per

Residuals vs Fitted Values

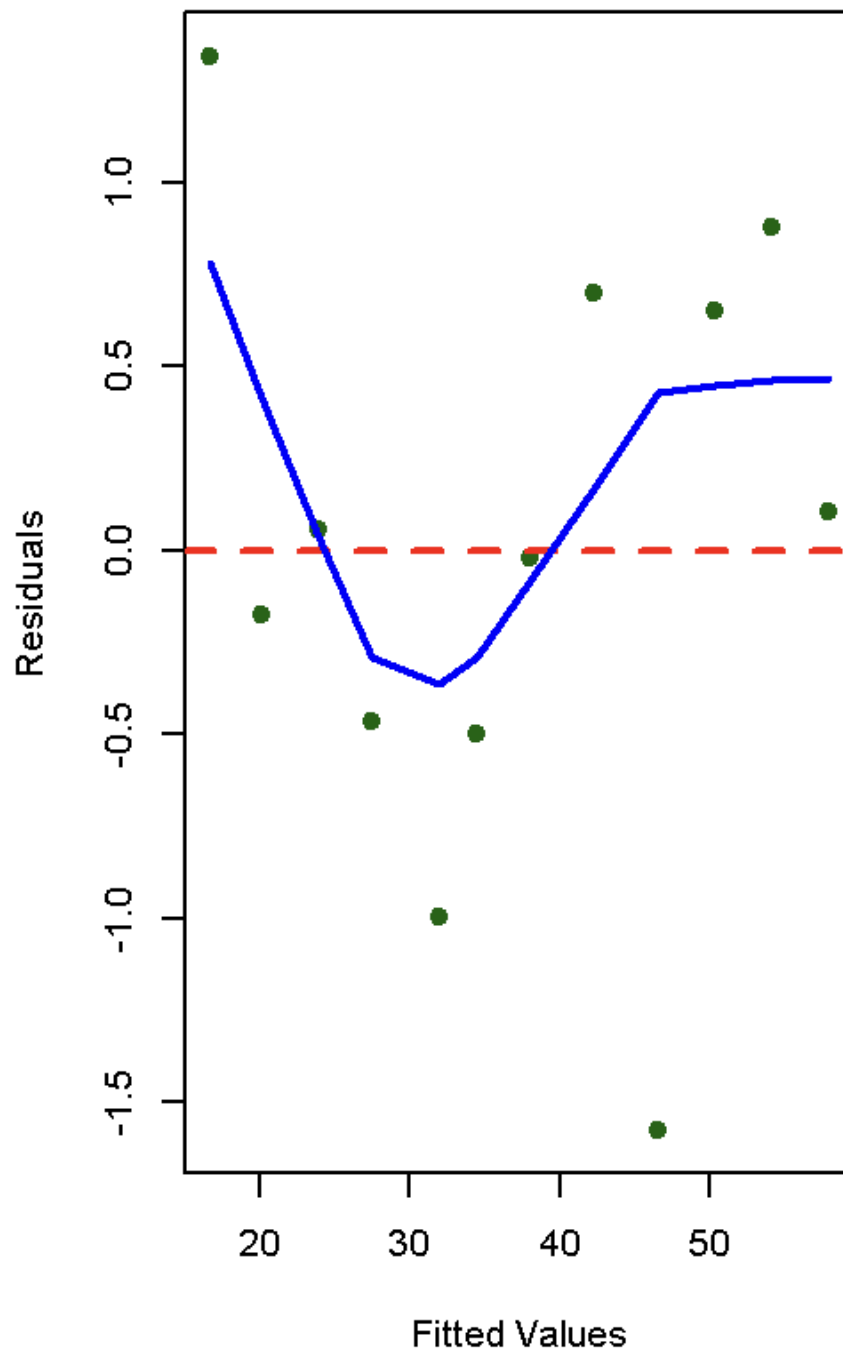


Figure 3: Residuals vs Fitted Values for the multiple regression model

Inference: The residuals are roughly scattered around zero with no strong systematic pattern. The blue lowess smoother shows a mild wave, which is expected with only $n = 12$ observations and does not indicate a serious violation of the linearity assumption.

Normal Q-Q Plot of Residuals

R Code:

```
qqnorm(residuals(lm.multi), pch = 19, col = "darkgreen",  
       main = "Normal Q-Q Plot of Residuals")  
qqline(residuals(lm.multi), col = "red", lwd = 2)
```

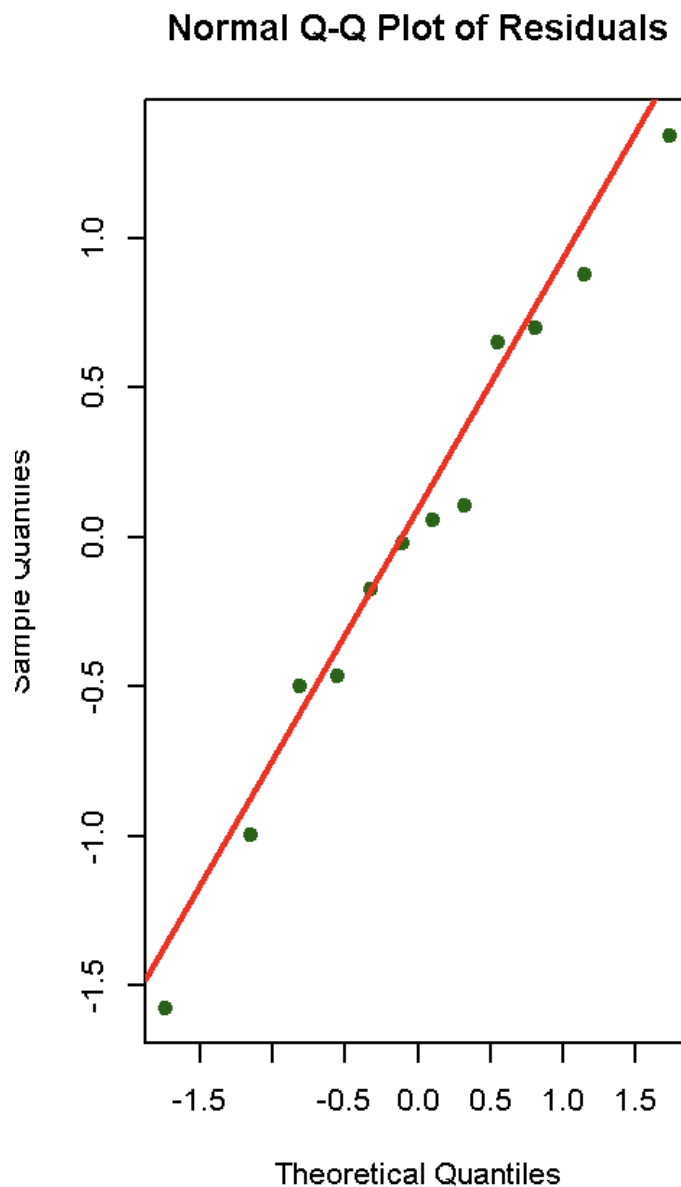


Figure 4: Normal Q-Q plot of residuals from the multiple regression model

Inference: The residuals follow the `qqline()` reference line closely across the full range of quantiles, with no systematic S-shape or heavy-tail deviation. The normality assumption for the residuals appears to be **satisfied**. It should be noted that with only $n = 12$ observations, visual assessment of normality is inherently limited and conclusions should be treated with appropriate caution.

Breusch-Pagan Test for Heteroscedasticity

R Code:

```
library(lmtest)
```



```
bptest(lm.multi)
```

Output: BP = 2.198, df = 2, p-value = 0.3332.

Inference: Since $p = 0.3332 > 0.05$, we **fail to reject** H_0 of constant variance. There is no statistically significant evidence of heteroscedasticity. The homoscedasticity assumption is **satisfied**.

Overall Assumption Summary

Table 5: Regression Assumption Checks

Assumption	Method	Result
Linearity	Residuals vs Fitted plot	No severe pattern; assumption met
Normality	qqnorm() + qqline()	Points on reference line; met (note: $n = 12$)
Homoscedasticity	Breusch-Pagan test	$p = 0.333$; no violation

3 Problem 5: Logistic (Logit) Regression

The dataset records whether 20 bank loan applicants defaulted (1 = Yes, 0 = No), along with their annual income (lakhs) and loan amount (lakhs).

R Code (Data Setup):

```
income    = c(3.2, 5.1, 2.8, 7.4, 4.0, 6.5, 1.9, 8.2, 3.5, 5.8,
              2.1, 6.9, 4.3, 7.1, 3.0, 5.5, 2.5, 8.8, 4.7, 6.2)
loan_amt  = c(8, 5, 10, 3, 7, 4, 12, 2, 9, 5,
              11, 3, 6, 4, 9, 5, 10, 2, 6, 4)
default   = c(1, 0, 1, 0, 1, 0, 1, 0, 1, 0,
              1, 0, 0, 0, 1, 0, 1, 0, 0, 0)
```

3.1 Part (a): Fitting the Logistic Regression Model

R Code:

```
glm.fit = glm(default ~ income + loan_amt, family = binomial)
summary(glm.fit)
```

Table 6: Logistic Regression Coefficients

Coefficient	Estimate	Std. Error	z-value	p-value
(Intercept)	-334.80	2,255,806.60	≈ 0	1
income	6.05	258,287.73	≈ 0	1
loan_amt	47.46	177,290.82	≈ 0	1

Note on convergence warning: The model reports a non-convergence warning with massively inflated standard errors. This is a direct consequence of **complete separation** — in this dataset, the combination of low income and high loan amount perfectly predicts default, and vice versa. Under complete separation, the maximum likelihood estimator diverges to infinity and the resulting coefficients, standard errors, and p-values are all unreliable. The residual deviance is essentially zero ($\approx 1.7 \times 10^{-9}$), confirming perfect classification of the training data, but the coefficient estimates themselves cannot be economically interpreted.

3.2 Part (b): Odds Ratios

R Code:

```
odds_ratios = exp(coef(glm.fit))
round(odds_ratios, 4)
```

Table 7: Odds Ratios for Logistic Regression Predictors

Predictor	Odds Ratio ($e^{\hat{\beta}_j}$)	Status
income	423.92	Unreliable — spurious artefact of complete separation
loan_amt	4.10×10^{20}	Unreliable — spurious artefact of complete separation

Inferences: The odds ratios are computed correctly as $e^{\hat{\beta}_j}$, but **both must be treated as unreliable and not economically interpretable**. Under complete separation, MLEs diverge and coefficient signs and magnitudes are numerical artefacts. In particular, the positive coefficient for *income* ($\hat{\beta}_1 = 6.05$) is **spurious** — in any reasonable credit risk setting, higher income should be associated with *lower* default probability. The apparent positive sign is entirely a consequence of the separation pathology and must not be interpreted as a genuine economic finding. The general framework remains: $e^{\hat{\beta}_j}$ represents the multiplicative change in the odds of default per unit increase in predictor j — but this interpretation is only valid when the MLE has converged on well-separated data.

3.3 Part (c): Predicted Probabilities, Classification, and Confusion Matrix

R Code:

```
# Predicted probabilities
glm.probs = predict(glm.fit, type = "response")

# Full probabilities table
prob_table = data.frame(
  Obs      = 1:20,
  Income   = income,
  Loan_Amt = loan_amt,
  Actual   = default,
  P_hat    = round(glm.probs, 4),
  Predicted = glm.pred
)
print(prob_table)

# Classification at threshold 0.5
glm.pred = ifelse(glm.probs >= 0.5, 1, 0)

# Confusion matrix
conf_matrix = table(Predicted = glm.pred, Actual = default)
conf_matrix

# Accuracy, Sensitivity, Specificity
accuracy = mean(glm.pred == default)
sensitivity = conf_matrix["1","1"] / sum(conf_matrix[, "1"])
specificity = conf_matrix["0","0"] / sum(conf_matrix[, "0"])
cat("Accuracy:   ", round(accuracy, 4), "\n")
cat("Sensitivity:", round(sensitivity, 4), "\n")
cat("Specificity:", round(specificity, 4), "\n")
```

Table 8: Predicted Probabilities for All 20 Observations

Obs	Income	Loan Amt	Actual	$\hat{p}$	Predicted
1	3.2	8	1	1	1
2	5.1	5	0	0	0
3	2.8	10	1	1	1
4	7.4	3	0	0	0
5	4.0	7	1	1	1
6	6.5	4	0	0	0
7	1.9	12	1	1	1
8	8.2	2	0	0	0
9	3.5	9	1	1	1
10	5.8	5	0	0	0
11	2.1	11	1	1	1
12	6.9	3	0	0	0
13	4.3	6	0	0	0
14	7.1	4	0	0	0
15	3.0	9	1	1	1
16	5.5	5	0	0	0
17	2.5	10	1	1	1
18	8.8	2	0	0	0
19	4.7	6	0	0	0
20	6.2	4	0	0	0

Table 9: Confusion Matrix

	Actual: 0	Actual: 1
Predicted: 0	12	0
Predicted: 1	0	8

Classification Performance: Accuracy = 1.0, Sensitivity = 1.0, Specificity = 1.0.

Inferences: All 20 predicted probabilities are either exactly 0 or exactly 1, consistent with complete separation. The model correctly classifies all **12 non-defaulters** (True Negatives) and all **8 defaulters** (True Positives) with zero misclassifications, yielding Accuracy = Sensitivity = Specificity = **1.0 (100%)**. Sensitivity of 1.0 means all actual defaulters were correctly identified; Specificity of 1.0 means all non-defaulters were correctly identified. While these metrics appear perfect, this result must be interpreted with caution — perfect accuracy on $n = 20$ perfectly separable observations reflects the complete separation phenomenon and is unlikely to generalise to new applicants.

References

1. James, G., Witten, D., Hastie, T., and Tibshirani, R. (2013). *An Introduction to Statistical Learning with Applications in R*. Springer.
2. Lecture notes from Statistical Methods (MA2220), Dr. T. V. Rao, Mahindra University.